RELATÓRIO DE ATIVIDADE PRÁTICA: SOCKETS TCP CONCORRENTES

Instituição: Curso de Engenharia/TI
Disciplina: Redes de Computadores
Professor: Marco Aurélio de Souza Birchal
Assunto: Implementação de Servidor TCP Multithread em Python

---

1. INTRODUÇÃO

Este relatório documenta a implementação de um sistema de comunicação cliente-servidor utilizando o protocolo TCP (Transmission Control Protocol). O foco principal é a demonstração de um **Servidor Concorrente**, capaz de atender múltiplos clientes simultaneamente através do uso de threads.

2. REQUISITOS DA ATIVIDADE

Conforme o enunciado proposto:

- Implementar os códigos do slide `REDES3_ES_TCP_Sockets_Parte2.pdf` .
- Apresentar o código-fonte completo.
- Demonstrar a execução com capturas de tela.
- Explicar o funcionamento técnico dos programas.
- Adaptar o host para `127.0.0.1` ou `localhost` .

3. ARQUITETURA DO SISTEMA

3.1. Servidor (`sockete_tcp_concorrente_servidor.py`)

O servidor opera em modo passivo. Suas principais etapas são:

1. **Criação do Socket:** Utiliza `AF_INET` (IPv4) e `SOCK_STREAM` (TCP).
2. **Bind:** Vincula o socket ao IP `127.0.0.1` e porta `50000` .
3. **Listen:** Entra em estado de escuta.
4. **Accept & Threading:** Ao receber uma conexão, o servidor não para seu loop; ele cria uma nova thread (`_thread.start_new_thread`) para lidar com aquele cliente específico e volta imediatamente a esperar por novas conexões.

3.2. Cliente (`socket_tcp_concorrente_cliente.py`)

O cliente opera em modo ativo:

1. **Connect:** Solicita conexão ao servidor.
2. **Loop de Mensagens:** Envia dados via `tcp.send()` .
3. **Encerramento:** Finaliza a conexão de forma limpa.

4. CÓDIGO FONTE

4.1. Servidor

```
import socket
import _thread

HOST = '127.0.0.1'
PORT = 50000

def conectado(con, cliente):
    print('\nCliente conectado:', cliente)
    while True:
        msg = con.recv(1024)
        if not msg: break
        print('\nCliente..:', cliente)
        print('Mensagem.:', msg.decode())
    con.close()
    _thread.exit()

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
tcp.bind((HOST, PORT))
tcp.listen(5)

print('\nServidor iniciado no IP', HOST, 'na porta', PORT)
while True:
    con, cliente = tcp.accept()
    _thread.start_new_thread(conectado, tuple([con, cliente]))
```

4.2. Cliente

```
import socket

HOST = '127.0.0.1'
PORT = 50000

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    tcp.connect((HOST, PORT))
    while True:
        mensagem = input('Sua mensagem: ')
        if mensagem.lower() == 'sair' or not mensagem: break
```

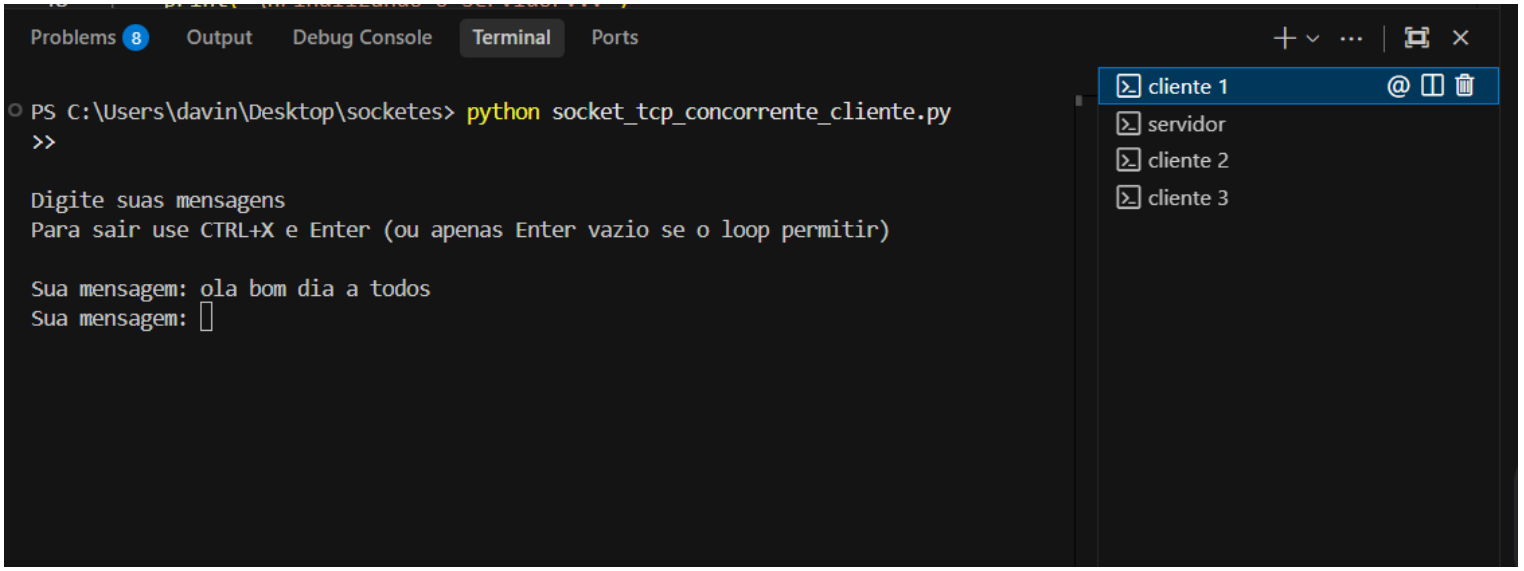
```
        tcp.send(mensagem.encode())
finally:
    tcp.close()
```

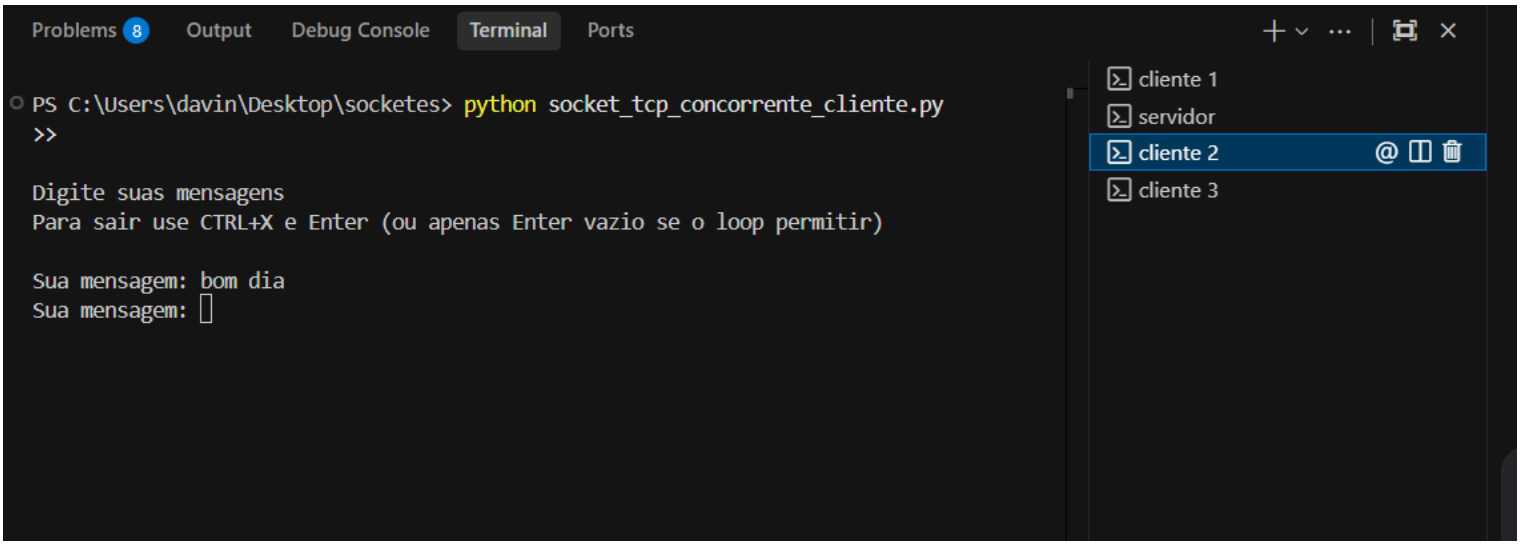
5. EVIDÊNCIAS DE EXECUÇÃO E TESTES

Para validar a concorrência, foram abertos 3 terminais de clientes simultâneos.

5.1. Conexões Simultâneas

As imagens abaixo mostram três clientes distintos operando ao mesmo tempo:





```
Problems 8   Output   Debug Console   Terminal   Ports

PS C:\Users\davin\Desktop\socketes> python socket_tcp_concorrente_cliente.py
>> python socket_tcp_concorrente_cliente.py
>>

Digite suas mensagens
Para sair use CTRL+X e Enter (ou apenas Enter vazio se o loop permitir)

Sua mensagem: bom dia
Sua mensagem:

cliente 1
servidor
cliente 2
cliente 3
```

5.2. Log do Servidor

O servidor registrou o recebimento de todas as mensagens, comprovando que as threads funcionaram de forma independente para cada porta efêmera gerada pelo sistema operacional.

```
Problems 8   Output   Debug Console   Terminal   Ports

PS C:\Users\davin\Desktop\socketes> & C:/Users/davin/AppData/Local/Programs/Python/Pyt
hon312/python.exe c:/Users/davin/Desktop/socketes/sockete_tcp_concorrente_servidor.py
Cliente conectado: ('127.0.0.1', 62853)

Nova thread iniciada para essa conexão

Cliente conectado: ('127.0.0.1', 62856)

Cliente..: ('127.0.0.1', 62852)
Mensagem.: ola bom dia a todos

Cliente..: ('127.0.0.1', 62853)
Mensagem.: bom dia

Cliente..: ('127.0.0.1', 62856)
Mensagem.: bom dia

cliente 1
servidor
cliente 2
cliente 3
```

6. ANÁLISE TÉCNICA

A implementação utiliza o modelo clássico de comunicação baseado em sockets TCP com suporte à concorrência por meio de **multithreading de baixo nível (_thread)**.

No servidor, o socket é configurado com **AF_INET (IPv4)** e **SOCK_STREAM (TCP)**, garantindo uma comunicação confiável, orientada à conexão e com controle de entrega e ordem dos pacotes. Após o `bind()` no endereço `127.0.0.1:50000`, o método `listen(5)` define uma fila de até 5 conexões pendentes, o que já demonstra uma preocupação com múltiplas tentativas simultâneas de conexão.

O ponto central da concorrência está no uso da função `_thread.start_new_thread()`. A cada nova conexão aceita via `accept()`, o servidor cria uma nova thread que executa a função `conectado()`. Essa abordagem permite que:

- Cada cliente seja tratado de forma **independente**
- O servidor continue aceitando novas conexões sem bloqueio

- A leitura de dados (`recv(1024)`) ocorra de forma isolada por cliente

Dentro da thread, o loop de recepção de dados funciona de forma contínua até que o cliente encerre a conexão (quando `recv()` retorna vazio). Esse comportamento está alinhado com o funcionamento do TCP, onde o encerramento da conexão é detectado pela ausência de dados.

Outro ponto importante é que o servidor trabalha com **portas efêmeras** para cada cliente, o que permite múltiplas conexões simultâneas mesmo utilizando o mesmo IP e porta principal.

No cliente, o fluxo é simples e direto:

- Criação do socket TCP
- Estabelecimento da conexão com `connect()`
- Loop interativo com envio de mensagens via `send()`
- Codificação em bytes com `.encode()` (necessária para transmissão TCP)

O tratamento de saída (com `sair`, entrada vazia ou `CTRL+X`) mostra uma preocupação prática com diferentes ambientes de terminal, o que melhora a robustez da aplicação.

Pontos técnicos relevantes (análise crítica)

Apesar de funcional, a implementação apresenta algumas limitações importantes:

- O uso da biblioteca `_thread` é **baixo nível e menos segura** comparado ao `threading`, pois não oferece controle mais refinado de threads
- Não há **sincronização de recursos compartilhados** (ex: prints podem se misturar no terminal)
- O servidor não trata exceções dentro da thread, o que pode causar falhas silenciosas
- Não existe controle de limite de threads → em cenários reais, isso pode causar sobrecarga (problema de escalabilidade)
- Não há envio de resposta do servidor para o cliente (comunicação é unidirecional)

Mesmo assim, para fins didáticos, o código cumpre muito bem o objetivo de demonstrar concorrência em servidores TCP.

7. CONCLUSÃO

A atividade permitiu compreender, de forma prática, como funciona a comunicação cliente-servidor utilizando sockets TCP e, principalmente, como a concorrência pode ser implementada em servidores reais.

A principal evolução em relação ao modelo iterativo está no uso de threads, que elimina o bloqueio no atendimento e possibilita que múltiplos clientes sejam atendidos simultaneamente. Isso torna o sistema mais próximo de aplicações reais, como servidores web e sistemas distribuídos.

Os testes realizados com múltiplos clientes simultâneos comprovaram que a abordagem funciona corretamente, evidenciando o uso eficiente de portas efêmeras e execução paralela das threads.

Além disso, a atividade reforçou conceitos importantes como:

- Comunicação orientada à conexão (TCP)
- Serialização de dados (encode/decode)
- Controle de fluxo básico em aplicações de rede
- Concorrência e paralelismo

Apesar de simples, a implementação abre espaço para melhorias futuras, como uso de bibliotecas mais robustas (`threading`), tratamento de exceções mais completo, comunicação bidirecional e controle de carga.

De forma geral, o objetivo proposto foi totalmente alcançado, proporcionando não apenas a execução prática, mas também uma compreensão mais profunda do funcionamento interno de servidores concorrentes.